Taller: señales en un servidor concurrente

Laboratorio de Sistemas Operativos

Objetivo: capturar señales en un servidor concurrente, programar una alarma periódica e informar los errores de las llamadas al sistema. Al terminar se cuenta con un servidor que termina de forma ordenada, presenta sus contadores y publica el estado del tablero cada cierto tiempo.

Planteamiento del problema

Una señal es una notificación asíncrona que el sistema operativo entrega a un proceso para indicarle que algo ocurrió. No transporta datos, ya que el proceso recibe el número de la señal y nada más, de modo que toda la información que el programa necesite debe estar ya en su propia memoria.

El servidor del tablero atiende a sus clientes dentro de un ciclo que no termina. La única manera de detenerlo consiste en interrumpirlo desde la terminal, y entonces el proceso desaparece sin cerrar el socket de escucha y sin informar cuántas operaciones alcanzó a atender. Las dos carencias se resuelven con el mismo mecanismo, debido a que la interrupción de la terminal llega al proceso como la señal `SIGINT`, y un programa puede instalar su propio manejador para ella.

Un manejador se ejecuta en medio de lo que el proceso estuviera haciendo, incluso dentro de una función de biblioteca a medio terminar. Por esta razón las operaciones permitidas dentro de un manejador forman un conjunto reducido, que no incluye `printf` ni la toma de un candado. Un manejador que tome el candado que el hilo interrumpido ya tenía produce un bloqueo del que el proceso no sale. La forma habitual de trabajar consiste en que el manejador registre el hecho en una variable y el ciclo principal haga el trabajo cuando la observe.

Este taller extiende el servidor con dos señales, la interrupción de la terminal y la alarma del temporizador, y con el informe de los errores de las llamadas al sistema. Los contadores que ese informe necesita son memoria compartida entre los hilos que atienden a los clientes, de manera que el problema de la sesión anterior reaparece sobre un dato nuevo.

Preparación

El directorio `src/` contiene el servidor, el cliente de carga, el `Makefile` y el guion `carga.sh` de la práctica anterior, sin modificaciones. El trabajo ocurre sobre `servidor_tablero.c`.

```
cd src
make
./servidor_tablero rwlock
```

Enunciado

1. Ejecutar el servidor sin modificaciones, conectar un cliente y detener el servidor con `Ctrl-C`. Registrar en la bitácora qué presenta el servidor al terminar y qué ocurre con el cliente que estaba conectado.

2. Declarar una bandera de tipo `volatile sig_atomic_t` e instalar un manejador para `SIGINT` con `sigaction`. El manejador asigna uno a la bandera y no realiza ninguna otra operación.

```
static volatile sig_atomic_t terminar = 0;

static void al_interrumpir(int senal) {
    (void) senal;
    terminar = 1;
}
```

   La instalación ocurre en `main`, antes de aceptar la primera conexión. El campo `sa_flags` queda en cero de forma deliberada, dado que la opción `SA_RESTART` reanudaría la llamada interrumpida y el ciclo no recuperaría el control.

```
struct sigaction accion;

memset(&accion, 0, sizeof(accion));
accion.sa_handler = al_interrumpir;
sigemptyset(&accion.sa_mask);
accion.sa_flags = 0;

if (sigaction(SIGINT, &accion, NULL) == -1) {
    perror("sigaction");
    return EXIT_FAILURE;
}
```

3. Convertir el ciclo de `accept` en un ciclo que termine cuando la bandera quede activa. Una llamada interrumpida por una señal retorna menos uno y deja en `errno` el valor `EINTR`, que el programa debe diferenciar de un error real. Al salir del ciclo, el servidor cierra el socket de escucha y presenta cuántas lecturas y cuántas escrituras atendió.

4. Proteger los dos contadores. Los hilos que atienden a los clientes los incrementan al despachar cada orden, de modo que un contador sin

protección repite el problema que la práctica anterior midió sobre el tablero. Registrar en la bitácora qué mecanismo se eligió y con qué criterio.

5. Instalar un manejador para `SIGALRM` y programar la alarma con `alarm`. Cada vez que el ciclo principal observe la bandera correspondiente, el servidor presenta la versión del tablero y su contenido, y vuelve a programar la alarma. Esa lectura del tablero aplica la política vigente en modo de lectura, igual que la orden `LEER` de un cliente.

6. Enviar las dos señales desde otra terminal con `kill` y comprobar el efecto de cada una. Intentar después capturar la señal que termina un proceso de forma incondicional, y explicar en la bitácora por qué el sistema no lo permite y en qué beneficia esa negativa.

```
./servidor_tablero rwlock &
kill -ALRM $!
kill -INT $!
kill -KILL $!
```

7. Comprobar el valor de retorno de las llamadas al sistema que el programa agrega e informar el error con `perror`. Provocar un fallo a propósito, arrancando un segundo servidor mientras el primero escucha, y registrar el mensaje que produce.

8. Explicar qué efecto tiene la línea `signal(SIGPIPE, SIG_IGN)` que el servidor ya contiene, y qué le ocurriría al servidor sin ella cuando un cliente cierra la conexión de golpe.

Condiciones de entrega

- El archivo `servidor_tablero.c` modificado, compilando sin advertencias con `gcc -Wall -Wextra`.
- La bitácora, con las respuestas de los puntos 1, 4, 6, 7 y 8 y con la salida de un informe periódico.

Criterios de valoración

- Que el manejador solo registre el hecho y el trabajo ocurra en el ciclo principal.
- Que el servidor termine cerrando el socket de escucha y presentando sus contadores.
- Que los contadores compartidos queden protegidos con un criterio justificado.
- Que los errores de las llamadas al sistema se informen y no se pasen por alto.

Errores frecuentes

- Llamar `printf` dentro del manejador. El programa funciona casi siempre, y esa es la razón de que el defecto sobreviva a las pruebas.
- Tomar dentro del manejador el candado que protege el tablero.
- Instalar el manejador con `SA_RESTART` y concluir que la señal no llega, cuando lo que ocurre es que `accept` se reanuda sin devolver el control al ciclo.
- Declarar la bandera como un entero corriente, sin `volatile sig_atomic_t`.
- Programar la alarma una sola vez y esperar informes periódicos.
- Tratar el retorno `EINTR` de `accept` como un error y abandonar el ciclo con un mensaje.